

CSS Flexbox — Master Revision Notes

Selva's hard copy · Lessons 54–57 (Display Flex · Flex Direction · Flex Layout · **Flex Sizing**) · full notes in [.github/my_tasks/concepts/](#)

🔥 The 8 lines that answer 90% of flexbox questions

1	display: flex goes on the CONTAINER . Its children's own block / inline rules are thrown away.
2	Main axis = the direction items flow. Cross axis = perpendicular. Both flip with flex-direction .
3	justify-content → main axis. align-items → cross axis. align-content → the wrap lines .
4	Sizing priority: content width < width < flex-basis < min/max-width . Clamping is last .
5	flex: <grow> <shrink> <basis> — "can I get bigger / can I get smaller / where do I start".
6	Default = flex: 0 1 auto → shrink YES, grow NO .
7	flex: 1 = 1 1 0% = equal columns . The one you'll type most.
8	grow/shrink are RATIOS , not pixels. 1 : 2 : 3 in 600px → 100 : 200 : 300 .

A The property map — parent or child?

Getting this wrong is the most common beginner mistake. Learn the two columns, not 12 properties.

🟣 CONTAINER (parent) properties

Property	Job
display	flex / inline-flex — switch it on
flex-direction	row * / row-reverse / column / column-reverse
flex-wrap	nowrap * / wrap / wrap-reverse
flex-flow	shorthand: direction wrap
justify-content	space along the main axis
align-items	position along the cross axis
align-content	space between wrap lines
gap	gutter between items (row-gap / column-gap)

🟠 ITEM (child) properties

Property	Job
order	visual position. Default 0 , low → high
flex-grow	share of spare space. Default 0
flex-shrink	share of the overflow . Default 1
flex-basis	starting size on the main axis. Default auto
flex	shorthand for the three above
align-self	override align-items for one item
margin: auto	eats free space → pushes items apart

* = default value. There is no justify-self / justify-items in flexbox — use margin: auto .

B Main axis vs cross axis

	flex-direction: row (default)	flex-direction: column
Main axis	horizontal →	vertical ↓
Cross axis	vertical ↓	horizontal →
justify-content	left ⇌ right	top ⇕ bottom
align-items	top ⇕ bottom	left ⇌ right
flex-basis sets	width	height
untouched	height still normal	width still normal

🔑 Why this matters

Every alignment property is defined **relative to the axes**, not to "left/right/top/bottom". Change flex-direction and justify-content: center silently switches from horizontal centering to **vertical** centering. Nothing else in your CSS changed.

C Turning it on — flex vs inline-flex

```
.container {  
  display: flex;      /* or inline-flex */  
  gap: 10px;         /* or 1rem - scales */  
}
```

	flex	inline-flex
Container box	full width (like block)	hugs content (like inline-block)
Line flow	own line	sits inline with text
Children	identical either way	

🧑 Your words — keep them

- "Rules meera padum." The second a parent goes `display: flex`, the children's own display rules are overruled and discarded. Flexbox is a **separate system** — stop reasoning about it with block/inline instincts.
- Normal `flex` = **100% block**. `inline-flex` = **inline-block**.
- Use `float` only for floating images around text — **NOT for layouts**.
- `list-style: none` kills the bullets when you flex a `<ul>` into a nav bar.

✅ Targeting the items

`.c div` = any depth · `.c > div` = direct child divs ·
`.c > *` = **every direct child, any tag** ← safest default.

D Layout — order, flex-wrap, justify-content

1 · order — rearrange without touching the HTML (child property)

- Every item defaults to `order: 0`. Flexbox sorts **lowest** → **highest**.
- Ties break by **HTML source order**.
- Negative values are allowed — `order: -1` jumps to the front.
- **Purely visual**. The DOM never moves, so screen readers and `Tab` order still follow the HTML. Don't use it to fix real content order.

```
.rainbow > .green { order: 1; } /* rest stay 0 */
```

`order: default 0 0 0 0` → then `.green { order: 1 }`

R G Y B

R Y B G

2 · flex-wrap — what happens when you run out of room (container property)

Value	Behaviour
<code>nowrap</code> (default)	Items squeeze to fit one line; once they can't squeeze more, they overflow off the page
<code>wrap</code>	Items that don't fit drop to a new line : top-left → bottom-right
<code>wrap-reverse</code>	Same, but new lines stack the other way : bottom-right → top-left

🧑 Your words

flex-wrap = word wrap in WordPad. Line's full → drop to the next one. It is a **CONTAINER** property — never on the child.

Watch the spelling: `nowrap` has **no hyphen**. `no-wrap` is silently invalid — the browser ignores it and you get the default anyway.

3 · justify-content — spacing along the MAIN axis (container property · needs leftover space to show up)

`flex-start` — packed at the start (DEFAULT)

A B C

`flex-end` — packed at the end

A B C

`center` — packed as a group, equal space each side

A B C

`space-between` — edges touch, NO space at the ends

A B C

`space-around` — equal space BOTH SIDES of each item (edges look half)

A B C

`space-evenly` — every gap identical, edges included

A B C

`flex-start:` [A][B][C]_____

`flex-end:` _____[A][B][C]

`center:` _____[A][B][C]_____

`space-between:` [A]_____[B]_____[C]

`space-around:` __[A]____[B]____[C]__

`space-evenly:` ___[A]___[B]___[C]___

🔑 space-around vs space-evenly — the exam question

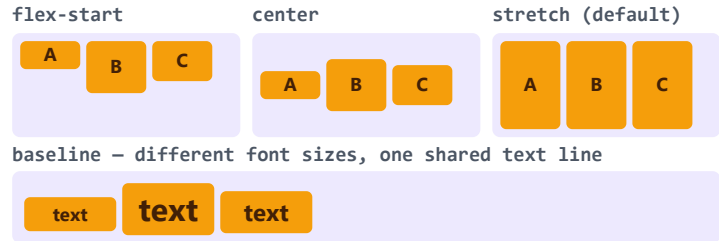
`space-around` gives every item an equal margin on **both of its own sides**. Between two items those two margins **add up**, so the middle gaps look **double** the edge gaps. `space-evenly` ignores that and just makes **every gap identical**.

E Layout — align-items, align-self, align-content

4 · align-items — position along the CROSS axis (container property)

In a `row` container the cross axis is **vertical**. It has nothing to work with until the container is **taller than its items** — give it a `height` (e.g. `70vh`) or you'll swear it's broken.

Value	Effect (row container)
<code>stretch</code> (default)	Items stretch to fill the container's height — <i>unless</i> they have a fixed height
<code>flex-start</code>	Aligned to the top
<code>flex-end</code>	Aligned to the bottom
<code>center</code>	Centred vertically
<code>baseline</code>	Items' text baselines line up, whatever the box sizes



⚠ flex-start vs baseline — the difference that catches people

`flex-start` lines up the **top edge of every box**, ignoring what's inside. `baseline` lines up the **invisible line the text sits on**, ignoring the box edges. They look identical **only** when every item shares the same font-size and padding — which is exactly why the difference stays hidden until it bites you on a real design.

5 · align-self — one item opts out (child property)

```
.container { display: flex; align-items: center; }
.container > .special { align-self: flex-start; }
/* only .special opts out of the centring */
```

Same five values as `align-items`, plus `auto` (= "just follow the container", the default). There is **no justify-self** in flexbox — for that job use `margin-left: auto`.

6 · align-content — spacing between WRAP LINES (container property)

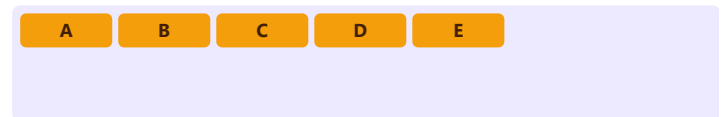
- Needs `flex-wrap: wrap` AND 2+ actual lines. On a single line it does **absolutely nothing** — this is the #1 reason it "doesn't work".
- Values: `flex-start` · `flex-end` · `center` · `space-between` · `space-around` · `space-evenly` · `stretch` (default).

🔑 The one-sentence difference

`align-items` moves items INSIDE a line. `align-content` moves the LINES THEMSELVES.

One line → only `align-items` can do anything. Many lines → both can.

`align-content: space-between` (2 wrap lines)



`align-content: center`



🔗 Alignment decision tree — read it top to bottom

I want to...	Use	Goes on
space items left→right (in a row)	<code>justify-content</code>	parent
move items up↑down (in a row)	<code>align-items</code>	parent
move just one item up/down	<code>align-self</code>	child
space the wrapped rows apart	<code>align-content</code> (+ <code>flex-wrap: wrap</code>)	parent
push one item to the far end	<code>margin-left: auto</code>	child
perfectly centre something	<code>justify-content: center</code> + <code>align-items: center</code>	parent

⚠ FLEX SIZING — the weak area

Slow down here. Everything below is one lesson: `flex-grow` / `flex-shrink` / `flex-basis`.

F The priority ladder — who overrides who

The course's own formula slide, left = weakest, right = strongest:

Content width the fallback — as wide as the text/image needs	width normal CSS size — ignored on the main axis once a basis exists	flex-basis the request — the size the item asks for	min-width / max-width the law — hard floor / hard ceiling, applied LAST
--	---	--	--

weakest ————— strongest — wins

🗣 Say it your way

The item **asks** (`flex-basis`) · the **law answers** (`min / max-width`) · `width` only speaks when `flex-basis` stays silent · and **content is the last man standing**.

F+ "Content width" is a RANGE, not a number

The piece that makes the rest click — **every element has TWO natural sizes**:

	What it is	The lesson's words
max-content the "ideal" width	all the text fits on ONE line	"all the content lined up, occupying the ideal estate"
min-content the floor	the width of the single longest WORD , at the current font-size	"the minimum width looks at the longest word"

min-content
(longest word)
|← item lives here →|
overflows
off-screen

max-content
(all on one line)
won't pass this
unless it can grow

With **no sizing properties at all**, an item starts at **max-content** and shrinks toward **min-content**.

🔑 Shrinking is NOT uniform

Each item has its **own** floor, because each has its own longest word. In the lesson's demo the box containing **"programming"** stays visibly widest — flexbox refuses to break a word across lines.

⚠ Past the floor it does NOT keep shrinking

It **overflows**: the content is "pushed off the page and will not be visible on screen anymore." This floor is exactly what `min-width: auto` resolves to (gotcha #1).

G flex-basis vs width — basis wins

```
.item {
  width: 300px;      /* ❌ ignored on main axis */
  flex-basis: 100px; /* ✅ the size used */
}
```

- If `flex-basis` is set, **width is completely ignored**. The lesson is blunt: "there's not even any point in setting it" — delete the `width`.
- If the basis **can't be honoured** (no room), the item falls back down the ladder to its **minimum content width** and stops there.

🔑 width and flex-basis behave IDENTICALLY under pressure

Whichever wins the lookup is only a **preferred** size — abandoned the moment there isn't room:

```
4 items × width:100px + gap:10px → needs ~430px
≥ 430px → every item is exactly 100px ✅
< 430px → the 100px is ABANDONED; each item
         shrinks toward its OWN min-content
```

⚠ "Main axis only" is the catch

Container	flex-basis controls	still normal
row (default)	width	height
column	height	width

In a column container, `width: 300px` is **not** ignored — `flex-basis` never fights the cross axis.

H flex-basis vs min-width / max-width — 4 scenarios, 1 rule

The browser sizes the item **first** (basis → grow/shrink), **then** clamps it. Clamping is the last step:

```
final size = clamp( min-width , size-from-basis/grow/shrink , max-width )
```

#	Setup	Does the basis respect the limit?	Final width	Why
S1	flex-basis: 200px + max-width: 100px	✗ breaks the ceiling	100px	basis overruled, clamped down
S2	flex-basis: 50px + max-width: 200px	✓ under the ceiling	50px	ceiling never used
S3	flex-basis: 200px + min-width: 300px	✗ breaks the floor	300px	basis overruled, clamped up
S4	flex-basis: 400px + min-width: 300px	✓ above the floor	400px	floor never used

🔑 The one rule for all four

If the requested size **already sits inside the allowed range**, flex-basis is the answer. If it falls **outside**, the **limit it broke becomes the answer**.

Shortcut: **vs max-width the smaller wins · vs min-width the larger wins**.

🧠 Limits are NOT targets

- **max-width = the potential to EXTEND** — how big it's *allowed* to get.
- **min-width = the potential to SHRINK** — how small it's *allowed* to get.
- Neither **asks** for anything. `max-width: 200px` will never stretch a 50px item to 200px.

⚠️ The trap you will fall into

Swap S4's min-width for max-width: 300px and the answer **flips to 300px** — that's just S1 again. **Always name the limit out loud — "ceiling or floor?" — before you answer.**

Two more: (1) max-width also caps flex-grow — the item grows into free space then stops dead, and the leftover goes to the others. (2) min-width **beats** max-width: min-width:300px; max-width:100px → **300px**, because the floor is checked last.

I The three levers

Property	The question it answers	Default	Unit
flex-grow	"If there's spare space, do I take a share of it?"	0 — no	ratio (unitless)
flex-shrink	"If we're overflowing, do I give up some of my size?"	1 — yes	ratio (unitless)
flex-basis	"What size do I start from, before any of that happens?"	auto	length / % / auto / 0

All three are **CHILD properties**. And grow/shrink are **ratios, not pixels** — `flex-grow: 5` means "give me 5 shares of the leftover while a 1 item gets 1 share", not "5px".

🔍 The 3 questions — steal the instructor's debugging method

"Resize the window and see how the green box behaves. **Does it grow? Does it shrink? What's the size that it wants to be?**"

Those three questions **are** the three properties. Run them on any layout you're copying or debugging **before writing a single line of CSS**.

Grab the window edge and ask...

Does it grow when I widen the window?	...you've found
Does it shrink when I narrow it?	flex-grow
What size does it want when there's room?	flex-shrink
	flex-basis

J The four grow/shrink scenarios — all with flex-basis: 100px

Only grow and shrink change. The whole **behaviour** of the item changes with them.

#	Shorthand	Grow?	Shrink?	Allowed range	The 100px acts as...
gs-1	flex: 0 0 100px	✗	✗	exactly 100px — constant	a fixed width. Zero flexibility
gs-2	flex: 1 0 100px	✓ freely	✗	100px → available space	a MINIMUM (a floor)
gs-3	flex: 0 1 100px	✗	✓ to min-content	min-content → 100px	a MAXIMUM (a ceiling) ← DEFAULT
gs-4	flex: 1 1 100px	✓	✓	min-content → available	only a STARTING LINE

gs-1	flex: 0 0 100px	[====100px====]		RIGID — a brick, whatever you do
gs-2	flex: 1 0 100px	[====100px====] →		100px is the FLOOR
gs-3	flex: 0 1 100px	← [====100px====]		100px is the CEILING ← the default
gs-4	flex: 1 1 100px	← [====100px====] →		100px is just the start line

🔑 **gs-3 is Flexbox's factory default**

flex: 0 1 auto is what every item does when you write **nothing at all**:

**"You are allowed to shrink.
You are NOT allowed to grow."**

That's exactly why a flex row **squeezes** when the window narrows but never **spreads out** to fill a wide one — until you hand out a flex-grow.

⚠️ **gs-4 and "DUMMY PIECE BAAVA" — the honest version**

Straight from the lesson: *"the flex-basis is pretty much completely ignored because flex-grow and flex-shrink are both on. It's going to grow to the max-width and shrink to the minimum width — and if those aren't set, it infers them from the content."* So the range becomes min-width/min-content ↔ max-width/available.

Calling it a dummy piece is the right instinct — but note the hedge "pretty much", and **one correction that matters two pages on**:

👉 **flex-basis is a dummy as a FINAL WIDTH. It is still very much alive as the STARTING LINE.** Free space is measured *after* every basis is laid down, then shared out. That's why flex: 1 1 0 and flex: 1 1 auto give **completely different** results.

K flex-basis: auto vs 0 — the switch you'll use most

Same grow, same shrink. Only the **starting line** moves — and that changes everything.

```
/* CONTENT PRIORITY — big content keeps room */
.item { flex: 1 1 auto; } /* share the LEFTOVER */

/* EQUAL COLUMNS — content is irrelevant */
.item { flex: 1 1 0; } /* share EVERYTHING */
```

flex: 1 1 auto — real measured widths, 600px row

short · 182px

a much longer sentence · 418px

flex: 1 1 0 — same markup, same container

short · 300px

a much longer sentence · 300px

	flex-basis: auto	flex-basis: 0
Starting size	each item's own content / width	0 for everyone
A 10-word sentence	gets maximum room	gets the same room as everyone
A single word	gets minimum room	gets the same room as everyone
Use it for	navbars, tag lists, "size me to my content"	equal columns , card grids, pricing tables

💡 **Mnemonic**

auto = "share the leftovers."
0 = "share everything."

If your columns "should be equal but aren't", the basis is auto when you wanted 0.

L The flex shorthand

```
flex: <flex-grow> <flex-shrink> <flex-basis>;

/* longhand */      /* shorthand */
flex-grow: 1;        flex: 1 1 0;
flex-shrink: 1;      /* shortened further */
flex-basis: 0;        flex: 1;
```

Shorthand	Expands to	Meaning
flex: 1	1 1 0%	equal columns. The workhorse
flex: 2	2 1 0%	takes 2 shares of the space
flex: auto	1 1 auto	grows & shrinks, content-weighted
flex: none	0 0 auto	rigid, sized to content
flex: initial	0 1 auto	the default: shrink yes, grow no
flex: 0 0 250px	—	a hard 250px sidebar

⚠ Correction to lock in — flex: 2 is NOT 2 2 0

A single number sets **flex-grow only**. flex-shrink stays at its default 1, and flex-basis becomes 0%.

~~flex: 2 → grow 2, shrink 2, basis 0~~
flex: 2 → grow 2, shrink 1, basis 0%

Verified with `getComputedStyle` in Chrome. It doesn't change the 1:2:3 result at all — with a basis of 0 there's **nothing to shrink** — but say it right in an interview.

👂 **Not a mis-hearing — the video says it too:** "in this case, it's a grow of 2 and a shrink of 2." A slip in the lesson. (flex: 1 → grow 1 / shrink 1 is right only because the default shrink already is 1.) **A single number never touches flex-shrink.**

⚠ The shorthand quietly resets the basis

```
.a { flex-grow: 1; } /* basis stays AUTO */
.b { flex: 1; }      /* basis becomes 0% */
```

.a gives **content-weighted** widths, .b gives **equal** ones. **Any omitted flex-basis in the shorthand defaults to 0, not auto.** This one line explains most "why aren't my columns equal?" bugs.

M Ratios in action — flex: 1 / 2 / 3

```
.one { flex: 1; }
.two { flex: 2; }
.three { flex: 3; }
```

- Total shares = 1 + 2 + 3 = 6. In a 600px container (basis 0, so all 600px is up for grabs) one share = 100px.
- The ratios are respected: 1x : 2x : 3x.**
- Resize and they **grow and shrink together, still 1:2:3** — the 3 item moves 3px for every 1px the 1 item moves. Add a 4th flex: 1 item → 7 shares → 600/7 each; the ratio rebalances by itself.

600px container • 6 shares • 100px per share

1 → 100px

2 → 200px

3 → 300px

same rule, narrower window — ratio unchanged

1

2

3

N How the browser actually computes it — the 5 steps

#	Step	What happens
1	Lay down the basis	Every item takes its flex-basis (or width, or its content size)
2	Measure free space	free = container - Σ(basis) - Σ(gaps) - margins
3	Distribute	Positive free space → share by flex-grow ratio . Negative (overflow) → claw back by flex-shrink × flex-basis
4	Clamp	Each item is squeezed into its min-width / max-width range
5	Position	justify-content places whatever space is still left over

💡 gap is eaten in step 2

It's subtracted **before** anything is shared. A 600px row with gap: 20px and 3 items only has **560px** to distribute → each flex: 1 item is **186.67px**, not 200px.

🔑 The asymmetry nobody mentions: shrink is weighted by basis

Grow is a plain ratio. **Shrink** is shrink × basis — a bigger item loses proportionally more. Verified: basis 200px + basis 400px squeezed into 300px → 100px and 200px, not 50px and 250px. *Makes sense — a big item can afford a bigger haircut.*

📌 Real-world gotchas — these *will* bite you in projects

🐛 #1 — "My item refuses to shrink" / "my text overflows" / "ellipsis doesn't work"

Every flex item silently carries `min-width: auto`, which resolves to its **min-content size** — the longest unbreakable word, or an image's intrinsic width. **flex-shrink cannot cross that floor.**

```
.item {
  flex: 1;
  min-width: 0;           /* ✅ THE FIX */
  /* overflow: hidden; also unlocks it */
}
```

Verified: a 100px container holding

Antidisestablishmentarianism stayed **190px** wide — refusing the container — until `min-width: 0` was added. Then it obeyed at **100px**.

This is the single most common flexbox bug in real code (long URLs, `<pre>` blocks, truncation that mysteriously fails).

🐛 #2 — box-sizing

`flex-basis` sizes the **content box** by default, so padding and border are added *on top*. Set `box-sizing: border-box` (usually globally on `*`) so `flex-basis: 200px` really means **200px total**.

🐛 #3 — Images don't shrink politely

They carry an intrinsic size. Use `min-width: 0 + max-width: 100%` to let them shrink, or `flex-shrink: 0` if you want them untouchable.

🐛 #4 — Percentage basis

`flex-basis: 50%` is a percentage of the **container's main size** — not of the item, not of the viewport. Handy: `flex: 1 1 50% + flex-wrap: wrap` = a 2-column responsive grid.

💡 #5 — margin: auto eats free space

`margin-left: auto` on one item pushes it (and everything after it) to the far end. The classic "logo left, links right" navbar trick — no justify-content gymnastics.

```
.navbar { display: flex; }
.navbar .login { margin-left: auto; }
```

💡 #6 — Debug with DevTools

In the Elements panel, every flex container has a little `flex` badge. Click it to overlay the axes, the gaps and the free space. The **Computed** tab shows the real resolved `flex-grow / shrink / basis` — the fastest way to catch a shorthand that reset your basis to `0`.

💡 #7 — Flexbox is 1-dimensional

It lays out a **row** or a **column**. When you need rows **and** columns to line up with each other in a real 2-D grid, that's **CSS Grid's** job — the next topic. They're partners, not rivals.

📌 The patterns you'll actually type

```
/* Equal-width columns */
.row > * { flex: 1; }

/* Fixed sidebar + fluid content */
.sidebar { flex: 0 0 250px; }
.content { flex: 1; min-width: 0; }

/* Holy grail: middle column twice as wide */
.left, .right { flex: 1; }
.middle { flex: 2; }
```

```
/* Responsive cards: min 250px, grow, wrap */
.cards {
  display: flex; flex-wrap: wrap; gap: 1rem;
}
.cards > * { flex: 1 1 250px; }

/* Perfect centring — the famous one */
.hero {
  display: flex;
  justify-content: center; /* main axis */
  align-items: center; /* cross axis */
  height: 100vh;
}
```

🔑 Sanity checklist when a flex layout misbehaves

1. Is `display: flex` on the **parent**? · 2. Which is the **main axis** right now? · 3. Is the property a **parent** or **child** one? · 4. Is there any **free space** for `justify-content` to distribute? · 5. Does the container have a **height** for `align-items`? · 6. Is `align-content` waiting on a **wrap**? · 7. Did a shorthand reset your **basis to 0**? · 8. Is `min-width: auto` blocking the shrink?

Rapid-fire self test

Cover the right-hand column. Target: all 20 in under 3 minutes, no notes.

#	Question	Answer	#	Question	Answer
1	width:300px; flex-basis:100px (row)	100px	11	Expand flex: 1	1 1 0%
2	flex-basis:200px; max-width:100px	100px	12	Expand flex: 2	2 1 0%
3	flex-basis:50px; max-width:200px	50px	13	Expand flex: none	0 0 auto
4	flex-basis:200px; min-width:300px	300px	14	The default flex value	0 1 auto
5	flex-basis:400px; min-width:300px	400px	15	flex: 1/2/3 in 600px	100/200/300
6	flex-basis:400px; max-width:300px	300px	16	flex-grow:1 vs flex:1	basis auto vs 0
7	min-width:300px; max-width:100px	300px — floor wins	17	Which property is applied last?	min/max-width
8	flex: 0 0 100px, window resized	100px always	18	Long word won't shrink — fix?	min-width: 0
9	flex: 1 0 100px, tiny window	100px — can't shrink	19	align-content does nothing — why?	no wrap / 1 line
10	flex: 0 1 100px, huge window	100px — can't grow	20	Which axis does align-items use?	cross

Q The course exercise — appbrewery.github.io/flexbox-sizing-exercise

Make the **blue** flexbox match the **green** one. Ideal widths **200 · 200 · 400px**. Run the 3 questions on each item *separately*.

Item	Grows?	Shrinks?	Wants to be
Item 1	no	yes	200px
Item 2	no	no	200px
Item 3	no	no	400px

```
.container { justify-content: space-between; }
.item1 { flex-basis: 200px; }
.item2 { flex-basis: 200px; flex-shrink: 0; }
.item3 { flex-basis: 400px; flex-shrink: 0; }
```

⚠ **The two traps**

1. The edge spacing isn't a sizing property — it's `justify-content: space-between` from the previous lesson. Always separate *"how are they spaced"* from *"how big are they"*.

2. flex-shrink: 0 is the only non-default here. Everything shrinks by default, so items 2 and 3 need it switched **off**. Nothing needs `flex-grow`.

R Write it from memory — the blank-page drill — once a week, blank sheet, no notes

1. The **priority ladder**, weakest → strongest, all four levels.

2. The **gs-1** → **gs-4** table: grow, shrink, allowed range, what the basis acts as. Circle the default.

3. All six **justify-content** values as ASCII sketches, plus all five **align-items** values.
4. The **five presets**: `flex: 1 / 2 / auto / none / initial` in longhand.

5. The **5-step algorithm**, and the one place `gap` is subtracted.

6. **Bonus**: the **two** content sizes — `max-content` and `min-content` — and which one is the floor.

S Keep practising

- 🎮 **Practice**

 - **Flexbox Froggy** — appbrewery.github.io/flexboxfroggy
24 levels, Intermediate difficulty (no hints)
 - **Flexbox Defense** — flexboxdefense.com
drills alignment under time pressure
 - **Flex Layout playground** — appbrewery.github.io/flex-layout
- 📖 **Reference**

 - **CSS-Tricks — Complete Guide to Flexbox** · the lookup sheet, splits parent vs child properties
 - **Josh Comeau — Interactive Guide to Flexbox** · best visual explanation of grow/shrink anywhere
 - **MDN — Controlling ratios of flex items** · the authoritative grow/shrink/basis page

🧑 **The five sentences to leave with**

1. **Container decides the layout, items decide their own size.**
2. **flex-basis asks. min / max-width decides. Clamping is always last.**
3. **Default = shrink yes, grow no.** Nothing spreads out until you hand out a `flex-grow`.
4. **flex: 1 means equal columns** because it secretly sets the basis to `0`.
5. **Grow and shrink are ratios, not pixels.** `1 : 2 : 3`, every time, at every window size.